larpix-lib on PACMAN

LArPix control and data stream software: setup, bench procedure, results and status

Maintained by the PACMAN agents (augie)

2026-11-02

1 Purpose and scope

This note records the work on `larpix-lib`, the new LArPix control software, with PACMAN and the zipline firmware and software (`zipline.tex`). The goal is for `larpix-lib` to replace (1) the control path, including readback for control, and (2) the data server; the focus is on (1). The ASICs themselves (bench setup, registers, TX settings) are recorded in `larpix.tex`.

2 Repository and environment

- `mulhearn/larpix-lib`, cloned in `/agents/cowork/larpix-lib` (shared repository; Michael pushes).
- Python environment: conda environment `larpix` per user, an editable install of the clone; `make test` runs the unit tests (27, OK).
- Network configuration: `config/network/single_bench.yaml`, the bench PACMAN at 10.42.0.175, port 5555 (`pacman_cmdserver`), `first_io_chan` 1, `uart_mask` 0xF. The demo `demos/init_network.py` uses it with the network `config/hydra/single.yaml` (chip 11, the FPGA on its west port).

3 IO channels

`pacman_cmdserver` sends a DATA word on channel n to UART $n-1$ and drops channel 0; on the RX side the firmware stamps each word with its UART's channel register (default UART + 1, writable), and the data server passes it through. The 1-based channel is an upstream convention, so it is not fixed in our code: `larpix-lib` takes the offset from `first_io_chan`, and the conversion stays in the PACMAN servers and firmware. With `first_io_chan` 0, `larpix-lib` routes channels 0–3 and cannot reach UART 3.

4 Bench procedure

1. Stop both servers; start `pacman_cmdserver` in the foreground; start `pacman_dataserver`, wait for its RX descriptor ring, and kill it (restarting the cmdserver resets the DMA and halts the RX channel; the data server rebuilds the ring). The data server must start after the cmdserver.
2. RX buffer enables to 0 (no heartbeats or markers; the cmdserver sets them at start).
3. Hardware check from `pacman_menu`: power, configuration root, hello; then all RX UARTs off and a full reset.

4. UART 0 on: hello unanswered (the chip ID is back to 1); UART 0 off.
5. Configure with `larpix-lib` (`demos/init_network.py`).
6. Hello from the menu on UART 0, without the menu's configuration root.

5 Configuration sequence

`init_network.py` on the single network writes 14 registers: the chip ID (to chip 1, the ID after reset: 122 = 0x0B), then to chip 11 the root termination (248 = 0x07, POSI1), the I/O settings (123, 239–244, 254, 255), the input enables (126 = 0x02, POSI1 only), no upstream outputs (124 = 0x00) and the downstream output to the FPGA (125 = 0x01, PISO0). All TX words go to channel 1 (`asic_config.tx_list`).

6 Results

Date	Board	Result
2026-11-02	1v5	First configuration by `larpix-lib` (`demos/init_network.py` from felix, through `pacman_cmdserver`), with the input enables left out: 13 writes. Then a menu hello without the configuration root: the reply exact, 3 of 3. Before it, after a full reset, hello had no reply.
2026-11-02	1v5	With the input enables written (126 = 0x02, POSI1 only; 14 writes): the reply exact, 3 of 3; read back, 124 = 0x00, 125 = 0x01, 126 = 0x02, 248 = 0x07. The same request on UARTs 1–3 unanswered, on UART 0 exact: UART 0 drives POSI1 (west), as `larpix-lib`'s port map has it.
2026-11-02	1v5	The full configuration sequence (with the input enables and `first_io_chan` 1; pushed as fce6523, "ASIC config working at test bench"): the reply exact, 3 of 3, the same read-back; through `larpix-lib`'s own routing, io channels 2–4 (UARTs 1–3) unanswered, channel 1 exact.

7 Open

- ASIC bring-up entirely from the remote server: from power-up to readout confirmation through `larpix-lib` alone, without `pacman_menu`.
- Graceful handling of test-stand deployment variations: the network file, the io channel offset, the TX channel fixed at 1 in `asic_config.tx_list`.
- For 3.7: separate config and data streams, with server and/or firmware support for matching read requests with their replies.